

OSSP-SKILL-WEEK 6

Session 6

Task 1 - Write a program to create and handle process escape sequences.

Demonstrate escaped spaces, special symbols, preserved characters, and validate the parser output using different inputs.

Source code:

```
GNU nano 8.7.1
#include <stdio.h>

int main() {
    char input[100];

    printf("Enter a string: ");
    fgets(input, sizeof(input), stdin);

    printf("\nOriginal Input: %s", input);

    printf("\nEscaped Characters:\n");
    printf("New Line   : \\n\\n");
    printf("Tab       : \\t\\n");
    printf("Backslash  : \\\\n");
    printf("Quote     : \\\"\\n");

    printf("\nParsed Output: ");

    for (int i = 0; input[i] != '\\0'; i++) {
        if (input[i] == '\\\\' && input[i + 1] == '\\n') {
            printf("\\n");
            i++;
        }
        else if (input[i] == '\\\\' && input[i + 1] == '\\t') {
            printf("\\t");
            i++;
        }
        else {
            printf("%c", input[i]);
        }
    }

    return 0;
}
```

Output:

```
user@LAPX-0934-890:~/session5$ mkdir session6
user@LAPX-0934-890:~/session5$ cd session6
user@LAPX-0934-890:~/session5/session6$ nano escape.c
user@LAPX-0934-890:~/session5/session6$ gcc escape.c -o escape
user@LAPX-0934-890:~/session5/session6$ ./escape
Enter a string: 3

Original Input: 3

Escaped Characters:
New Line   : \n
Tab        : \t
Backslash  : \
Quote      : \"

Parsed Output: 3
```

Observation:. The program successfully demonstrates the handling of escape sequences such as newline, tab, backslash, and quotation marks. It reads the input string, identifies the escape characters, and produces the corresponding parsed output correctly.

Task 2: Write a program to create child processes and execute programs.

Demonstrate execution errors, passing arguments, managing the parent process, and command launching.

Source code:

```

GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid;

    printf("Parent process started.\n");

    pid = fork();

    if (pid < 0) {
        printf("Error: Child process creation failed.\n");
    }
    else if (pid == 0) {
        printf("Child process created successfully.\n");
        printf("Child Process ID: %d\n", getpid());

        execlp("ls", "ls", "-l", NULL);

        printf("Execution failed.\n");
    }
    else {
        printf("Parent Process ID: %d\n", getpid());
        wait(NULL);
        printf("Child process completed.\n");
    }

    return 0;
}

```

Output:

```

user@LAPX-0934-890:~/session5/session6$ nano child.c
user@LAPX-0934-890:~/session5/session6$ gcc child.c -o child
user@LAPX-0934-890:~/session5/session6$ ./child
Parent process started.
Parent Process ID: 261
Child process created successfully.
Child Process ID: 262
total 40
-rwxr-xr-x 1 user user 16160 Sep 15 06:58 child
-rw-r--r-- 1 user user 610 Sep 15 06:58 child.c
-rwxr-xr-x 1 user user 16176 Sep 15 06:57 escape
-rw-r--r-- 1 user user 732 Sep 15 06:57 escape.c
Child process completed.
user@LAPX-0934-890:~/session5/session6$

```

Observation: The program successfully creates a child process using `fork()`. The child process executes the given command using `execlp()`, while the parent process waits for the child process to complete using `wait()`. The program also displays the process IDs and handles process creation errors.